



EAST WEST UNIVERSITY

Department of Computer Science and Engineering

Semester: Fall 2023

Course Code: CSE 207

Course Title: Data Structures

Section: 7

Lab Manual: 01

Submitted to

Md. Manowarul Islam

Adjunct Faculty, Department of CSE

Submitted by

Swarna Rani Dey

2022-1-60-340

Submitted On

13.12.2023

Exercise 1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define INITIAL_CAPACITY 5
```

```
int *Array;
```

```
int top = -1;
```

```
int capacity = INITIAL_CAPACITY;
```

```
void ResizeArray() {
```

```
    capacity *= 2;
```

```
    Array = (int *)realloc(Array, capacity * sizeof(int));
```

```
}
```

```
void Push() {
```

```
    int n;
```

```
    if (top == capacity - 1) {
```

```
        ResizeArray();
```

```
}
```

```
printf("\nEnter the number: ");  
scanf("%d", &n);
```

```
top = top + 1;  
Array[top] = n;  
}
```

```
void Display() {  
    if (top == -1) {  
        printf("\nStack is empty\n");  
    } else {  
        printf("\nStack elements:\n");  
        for (int i = top; i >= 0; i--) {  
            printf("%d\n", Array[i]);  
        }  
    }  
}
```

```
int Pop() {  
    int value;  
    if (top == -1) {  
        printf("\nStack underflow\n");  
        return -1;  
    } else {
```

```
    value = Array[top];  
    top = top - 1;  
    return value;  
}  
}
```

```
int main() {  
    Array = (int *)malloc(INITIAL_CAPACITY * sizeof(int));  
  
    if (Array == NULL) {  
        printf("Memory allocation failed\n");  
        return -1;  
    }  
}
```

```
int option;  
while (1) {  
    printf("\n1. Push to Stack\n");  
    printf("2. Print Stack\n");  
    printf("3. Pop from Stack\n");  
    printf("4. Exit\n");  
    printf("Please enter your choice: ");  
    scanf("%d", &option);  
  
    switch (option) {  
        case 1:
```

```
        Push();
        break;
case 2:
    Display();
    break;

case 3: {
    int poppedValue = Pop();
    if (poppedValue != -1) {
        printf("\nPopped value: %d\n", poppedValue);
    }
    break;
}

case 4:
    free(Array);
    return 0;
default:
    printf("\nInvalid choice. Please try again.\n");
}
}

return 0;
}
```

OUTPUT

```
Output
Clear

/tmp/rcH4kwymMt.o
1. Push to stack
2. Print stack
3. Pop from stack
4. Exit
Please enter your choice: 1
Enter the number: 40
1. Push to stack
2. Print stack
3. Pop from stack
4. Exit
Please enter your choice: 2
Stack elements:
40

1. Push to stack
2. Print stack
3. Pop from stack
4. Exit
Please enter your choice: 3
Popped value: 40

1. Push to stack
2. Print stack
3. Pop from stack
4. Exit
Please enter your choice: 4
Exiting program
```

Exercise 2

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#define MAX 5
```

```
int Array[MAX];
```

```
int top = -1;
```

```
int isFull() {  
    return top == MAX - 1;  
}
```

```
int isEmpty() {  
    return top == -1;  
}
```

```
void push(char value) {  
    if (isFull()) {  
        printf("\nStack overflow\n");  
    } else {  
        top++;  
        Array[top] = value;  
    }  
}
```

```
char pop() {  
    if (isEmpty()) {  
        printf("\nStack underflow\n");  
        return '\0'; // Returning a null character for an empty  
stack  
    } else {
```

```
        return Array[top--];
    }
}
```

```
void display() {
    printf("\nStack contents:\n");
    for (int i = top; i >= 0; i--) {
        printf("%c\n", Array[i]);
    }
}
```

```
int main() {
    int flag = 0;
    char input[50];

    printf("\nEnter your expression : ");
    scanf("%s", input);

    for (int i = 0; i < strlen(input); i++) {
        switch (input[i]) {
            case '(':
                push(input[i]);
                break;
        }
    }
}
```



```
        case ')':
            if (pop() != '(') {
                flag = 1;
            }
            break;
    }
}

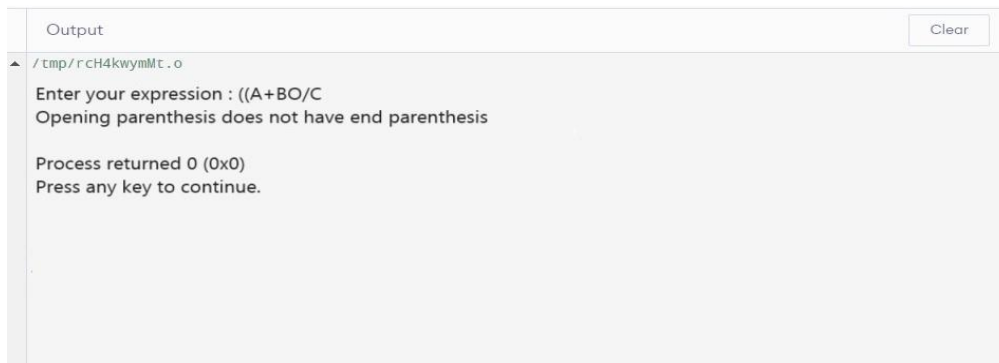
if (flag == 1) {
    printf("\nClosing parenthesis does not have opening
parenthesis\n");
}

if (!isEmpty()) {
    printf("\nOpening parenthesis does not have a matching
closing parenthesis\n");
}

display();

return 0;
}
```

OUTPUT



```
Output
/tmp/rcH4kwymlt.o
Enter your expression : ((A+BO/C
Opening parenthesis does not have end parenthesis

Process returned 0 (0x0)
Press any key to continue.
```

Exercise-3

```
#include<stdio.h>
```

```
int isParenthesisValid(char exp[]) {
```

```
    int count = 0, i = 0;
```

```
    while (exp[i] != '\0') {
```

```
        if (exp[i] == '(' || exp[i] == '[' || exp[i] == '{') {
```

```
            count++;
```

```
        } else if (exp[i] == ')' || exp[i] == ']' || exp[i] == '}') {
```

```
            count--;
```

```
            if (count < 0) {
```

```
                return 0;
```

```
            }
```

```

        }
        i++;
    }

    return (count == 0) ? 1 : 0;
}

int main() {
    char exp[50];
    printf("\nEnter an expression: ");
    scanf("%s", exp);

    if (isParenthesisValid(exp)) {

        printf("The parenthesis is valid\n");
    } else {
        printf("The parenthesis is invalid\n");
    }

    return 0;
}

```

OUTPUT

```
Output Clear  
▲ /tmp/8w5Kz8f0tC.o  
Enter an expression: ()(  
The parenthesis is invalid  
|
```

Exercise -4

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int Array[MAX];
```

```
int top = -1;
```

```
int isEmpty() {  
    return top == -1;  
}
```

```
int isFull() {  
    return top == MAX - 1;  
}
```

```
int pop() {
```

```
    if (!isEmpty()) {  
  
        return Array[top--];  
    } else {  
        printf("Stack is empty.\n");  
        return -1;  
    }  
}
```

```
void push(int value) {  
    if (!isFull()) {  
        Array[++top] = value;  
    } else {  
        printf("Stack is full.\n");  
    }  
}
```

```
int main() {  
    int num, remainder;  
  
    printf("Enter a decimal number: ");  
    scanf("%d", &num);
```

```
while (num >= 1) {  
    remainder = num % 2;  
    push(remainder);  
    num = num / 2;  
}  
  
printf("Binary equivalent: ");  
while (!isEmpty()) {  
  
    int bit = pop();  
    printf("%d", bit);  
}  
  
return 0;  
}
```

OUTPUT

Output

Clear

▲ /tmp/8w5Kz8f0tC.o
Enter a decimal number: 4
Binary equivalent: 100